

Beach Sand Mineral Image Classification

Reproduction and Critical Evaluation of “*D-ResNet: deep residual neural network for exploration, identification and classification of beach sand minerals*” (Theerthagiri & Prasannavenkatesan, 2022)

A from-scratch pipeline: exploratory data analysis, data wrangling, leakage-safe preprocessing, four-architecture training, and a 24-trial hyperparameter study

Item	Detail
Dataset	Kaggle: prasannavenkatesant/beach-sand-mineral-bsm (367 images, 6 classes)
Models	VGG16, ResNet18, ResNet34, D_ResNet (torchvision ResNet50 approximation)
Split	256 / 55 / 56 source images → 3072 / 55 / 56 samples
Hardware	NVIDIA Tesla T4 (Colab / Kaggle), CUDA, mixed precision
Training	60 epochs baseline; 20 epochs × 24 tuning trials
Total GPU time	56 min (baseline, 4 models) + 1.81 h (24 tuning trials)
Headline finding	Regularisation raised test accuracy but not beyond seed noise

Honest headline. Our best baseline model reaches **0.8036** test accuracy. The paper reports **0.89** for its proposed D-ResNet. We do not claim to match the paper, and Section 10 sets out exactly why the comparison is not like-for-like before concluding that the gap cannot be attributed to our models alone. This report deliberately presents the result that contradicts the expected narrative.

1. Executive summary

We reproduced the experimental setting of a published beach-sand-mineral classification study using four ImageNet-pretrained convolutional networks on a 367-image, six-class microscope dataset. The project produced two distinct bodies of work:

- **Baseline (notebooks 01–03).** Full EDA, leakage-safe preprocessing, 60-epoch training of four architectures, and a comparative analysis.
- **Tuning study (notebooks 04–05).** 24 trials — two configurations × three seeds × four architectures — to measure how much of the baseline result is attributable to the configuration rather than to seed luck.

The tuning study changed our interpretation of the baseline. Repeating the *identical* configuration across three seeds produced test-accuracy standard deviations of 0.018 to 0.047. Since one test image is worth 1.8 percentage points, the entire spread between the strongest and weakest architecture is comparable to two or three images of noise. **The baseline ranking was not resolvable from a single seed**, and the two highest reported figures both sat above their own three-seed means — favourable draws rather than typical outcomes.

A second finding concerns the tuning result itself. Regularisation (weight decay $1e-4$ + label smoothing 0.1) raised mean test accuracy from 0.7321 to 0.7708 (+0.0387) while mean validation accuracy moved only $0.8455 \rightarrow 0.8530$ (+0.0075). Because validation and test disagreed by roughly six to one, **we do not claim regularisation improved the models**. The test movement is most plausibly seed variance. Section 9 documents this in full.

Deliverables.

Notebook	Purpose	Training?
01_Data_Preprocessing_and_EDA	EDA, wrangling, augmentation, source-group split	No
02_Model_Experiments	GPU check, smoke test, 4 models × 60 epochs	Yes
03_Final_Comparison	Reads CSVs; comparison and discussion	No
04_Tuning	24 trials: 2 configs × 3 seeds × 4 models	Yes
05_Tuning_vs_Baseline	Reads outputs; verdict on tuning vs baseline	No

All five notebooks are committed in executed form, with outputs intact.

2. Dataset description and exploratory data analysis

2.1 Source and composition

The dataset is **beach sand mineral (BSM)** microscope imagery, obtained from Kaggle (prasannavenkatesant/beach-sand-mineral-bsm, public) with a Mendeley Data mirror (10.17632/9kyrz3shk8). It contains microscope photographs of six heavy-mineral grains:

- **Ilmenite** — FeTiO_3 , common accessory mineral of sands
- **Rutile** — TiO_2 , high refractive index, bright under reflected light
- **Zircon** — ZrSiO_4 , resistant, often rounded
- **Garnet** — dodecahedral silicate, common in beach placer deposits
- **Monazite** — $(\text{Ce}, \text{La}, \text{Nd}, \text{Th})\text{PO}_4$, phosphate bearing rare earths
- **Sillimanite** — AlSiO_3 , aluminosilicate polymorph

Classes are highly imbalanced in the raw dataset only by a small margin; the six counts range from 59 to 66, so no class dominates and accuracy is a reasonable primary metric. We nevertheless report macro-averaged precision, recall, F1 and specificity throughout, because macro-averaging weights every class equally and cannot be inflated by a majority class.

2.2 Dataset discrepancy against the paper

The paper's text states **357** images, but its own six per-class counts sum to **367**. We report what is actually on disk and never force a total to match the paper. Several class labels also appear misassigned in the source relative to the paper's counts.

Class	Actual on disk	Paper-reported	Difference
Ilmenite	62	60	+2
Rutile	66	62	+4
Zircon	59	60	-1
Garnet	60	66	-6
Monazite	60	60	+0
Sillimanite	60	59	+1
TOTAL	367	367	+0
TOTAL	367	367	0

Table 2.1 — Class distribution. The paper's text total (357) disagrees with its own class counts (367). Our pipeline uses the on-disk truth.

2.3 EDA performed

Exploratory analysis was carried out in notebook 01 and is reproduced as CSV and PNG artefacts under `results/dataset_summary/`:

EDA step	Finding	Artefact
Full file inventory	367 files, all readable, 0 corrupt	raw_dataset_inventory.csv
Class distribution	Near-balanced, 59–66 per class (Table 2.1)	dataset_summary.csv
Image dimensions	Variable; minimum side is the limiting factor for resize	image_dimensions.png
Visual sample grid	Background differs systematically by class	sample_images.png
Duplicate detection	0 exact duplicates; 367 unique content hashes	data_wrangling_report.csv

The single most important EDA finding is a methodological one. Visual inspection showed that background composition correlates with class — a Garnet field is visibly pink, an Ilmenite field grey-black. A CNN can reach high accuracy by learning background colour instead of grain morphology. A separate audit of this dataset quantified the association at Cramer's $V \neq 0.442$. We therefore treat **every accuracy figure in this report as an upper bound** on genuine mineral identification, and say so rather than presenting a number we know is inflated. This confound was not re-measured inside this project, so it is reported as a documented risk carried from prior analysis, not as a new result.

3. Data preprocessing and leakage control

3.1 Wrangling outcome

Validation found no unreadable files and no exact duplicates, so **0 images were removed**. All 367 were retained. The wrangling step exists mainly to prove that nothing was lost, which the version-fingerprint table records.

3.2 Augmentation

Following the paper, each training source was expanded by **6 rotations (0°, 30°, 45°, 90°, 180°, 270°) × 2 flip states** (original and horizontally flipped) = 12 variants per source, giving **3072 augmented training samples** from 256 sources.

3.3 The leakage safeguard — our deliberate deviation from the paper

The paper augments first and splits afterwards. That ordering places rotated and flipped **copies of the same photograph** across train, validation and test. A model can then score well by memorising one grain rather than learning to classify minerals, and the reported test accuracy measures memorisation instead of generalisation.

Safeguard	Paper behaviour	This implementation
Split ordering	Augment, then split	Split first at source-image level , then augment train only
Test set content	Augmented test set	Clean, non-augmented test set
Leakage check	Not reported	Automated, PASSED , re-verified at train time

Table 3.1 — Deviation from the paper. This makes our test set strictly harder than the paper's, which is central to the comparison in Section 10.

Splitting at the **group level** (one group per original photograph) means all 12 augmented variants of a photograph stay together in the train split. The split is 70/15/15 at source-image level with seed 42, yielding:

Split	Source images	Samples	Augmented?
Train	256	3072	Yes (12×)
Validation	55	55	No
Test	56	56	No

Table 3.2 — Final split. Validation and test contain only clean, un-augmented source images.

The leakage check was automated rather than assumed, and re-run independently at the start of notebook 02 to catch any drift between notebooks. It asserts that train source IDs are disjoint from validation and test source IDs, and that no validation or test sample is an augmented one. It reports **PASSED** in the version summary.

3.4 Reproducibility fingerprint

Notebook 01 writes a SHA-256 fingerprint of each manifest plus the full library and environment versions, so any later change to the data is detectable:

Item	Recorded value
Raw inventory SHA-256	967a983713f86dc5d455e8b10f57732a...
Final split manifest SHA-256	aaf9657bd54234bab042a5c7ac133ce6...
Augmentation manifest SHA-256	6017c25b23483923789c491f34e4a1ce...
Python / NumPy / pandas / Pillow / scikit-learn	3.13.15 / 2.1.3 / 2.2.3 / 11.3.0 / 1.6.1
Split seed	42
Leakage check	PASSED - no group spans splits; no augmented val/test samples

Table 3.3 — Dataset fingerprint (full values in results/preprocessing/dataset_version_summary.csv).

4. Model architectures, parameters and training configuration

4.1 Architectures and exact parameter counts

All four architectures come from torchvision with ImageNet-pretrained weights. Only the final classifier is replaced with a 6-output linear layer. **No architecture is hand-implemented.** Parameter counts below are trainable parameters, verified at runtime by the model-construction step in notebook 02.

Model (our label)	torchvision arch	Trainable parameters	Classifier layer
VGG16	vgg16	134.29 M	134290,000
ResNet18	resnet18	11.18 M	11180,000
ResNet34	resnet34	21.29 M	21290,000
D_ResNet	resnet50	23.52 M	23520,000

Table 4.1 — Architectures and parameter counts. VGG16 is by far the largest (134.29 M) and also the slowest to train (25.8 s/epoch vs 7.1 for ResNet18) — its three fully-connected layers dominate.

Important disclosure on D_ResNet. The paper proposes D-ResNet, described as a 50-layer bottleneck residual architecture. **The original source implementation and all training hyperparameters are not publicly available.** We therefore use torchvision ResNet50 as a practical architectural approximation. **This is not an exact reproduction of D-ResNet** and no result in this report should be read as one. This single substitution is the largest reason our D_ResNet row cannot be compared directly with the paper's D-ResNet row, and it is the reason its comparison appears in Section 10 with that caveat attached.

4.2 Hyperparameters, stated explicitly

Parameter	Baseline value	Tuning study value	Basis
Epochs	60 per model (240 total, 4 models)	20 per trial (480 total, 24 trials)	Paper specifies epochs 1–60
Batch size	32	32	Engineering choice
Optimiser	Adam	Adam	Engineering choice
Learning rate	1e-4 (fixed, no scheduler)	1e-4 (fixed, no scheduler)	Engineering choice
Weight decay	0.0	0.0 (A) / 1e-4 (B)	Tuned
Label smoothing	0.0	0.0 (A) / 0.1 (B)	Tuned
Loss	CrossEntropyLoss	CrossEntropyLoss(label_smoothing=S)	Engineering choice
Seeds	42 (single)	42, 43, 44	Reproducibility / variance
Image size	224 × 224 × 3 RGB	224 × 224 × 3 RGB	Pretrained-weight requirement
Normalisation	ImageNet mean/std	ImageNet mean/std	Pretrained-weight requirement
BatchNorm / Scheduler	As in torchvision; no scheduler	As in torchvision; no scheduler	—
Mixed precision	AMP enabled (eval forced FP32)	AMP enabled (eval forced FP32)	Stability
Split	256 / 55 / 56 sources	identical (reused manifest)	Leakage safeguard
Checkpoint rule	Best validation accuracy	Best validation accuracy	Never selected on test

Table 4.2 — Full hyperparameter specification. Note the baseline uses zero weight decay and zero label smoothing: it is a completely unregularised model, which is what motivated the tuning study.

4.3 Epoch budget per model

The paper specifies epochs 1–60, so the baseline was run for the full **60 epochs per model** and all four models were completed: **240 epochs in total**, 56 minutes wall-clock on one T4.

Model	Epochs run	Best epoch	Total time	Per epoch	Best val acc
ResNet18	60	30	7.1 min	7.1 s	0.8727
VGG16	60	4	26.2 min	25.8 s	0.8182
ResNet34	60	2	9.2 min	9.2 s	0.8364
D_ResNet	60	8	13.5 min	13.5 s	0.8364

Table 4.3 — Epoch accounting. All four models completed 60/60 epochs. The 'best epoch' column reveals that the useful training finished far earlier: three of four peaked at epoch 30 or earlier, and ResNet34 peaked at epoch 2.

Training accuracy reached approximately **1.000 by epoch 2** (VGG16 by epoch 6) and stayed there for the remaining epochs. With only 256 unique training photographs, the models memorised the training set within a few epochs. The 60-epoch budget was run in full as the paper specifies, but the operative operating point is epoch 2–30. This observation directly justified using **20 epochs** for the tuning trials: there was no reason to spend 3× the compute re-confirming convergence.

4.4 Engineering safeguards

- **GPU enforced.** Training aborts before the full run if `torch.cuda.is_available()` is False, so four models can never be silently trained on CPU.
- **Smoke test first.** A 1-epoch / 12-sample run of ResNet18 exercises dataset loading, transforms, dataloader, forward pass, metrics, plotting and checkpoint writing before the 56-minute run begins. It completed in 3.9 s.
- **FP32 evaluation.** Under AMP, VGG16's evaluation forward pass produced NaN. Evaluation is therefore always forced to FP32, while training uses AMP.
- **Non-finite checkpoint guard.** A checkpoint is only saved when validation loss is finite and validation accuracy improves, so a diverged run cannot leave a misleading 'best' weight file.

5. Metrics and evaluation protocol

Four metric families are reported per class and macro-averaged. Specificity is derived from the confusion matrix as one-vs-rest $TN/(TN+FP)$, because scikit-learn does not provide it directly.

Metric	Meaning	Why it matters here
Accuracy	Fraction of test images correctly classified	Primary metric; equals macro recall on a balanced set
Precision (macro)	Average of per-class $TP/(TP+FP)$	Detects a model predicting a class too eagerly
Recall / sensitivity (macro)	Average of per-class $TP/(TP+FN)$	Detects missed minority classes
F1 (macro)	Harmonic mean of the above	Single balanced summary
Specificity (macro)	Average one-vs-rest $TN/(TN+FP)$	Complements sensitivity; high specificity with low recall means a model that predicts only one class

Table 5.1 — Metric definitions. Macro-averaging is used throughout so that no class can dominate the score.

Model selection never used the test set. Within each run, the checkpoint kept is the one with the highest **validation** accuracy; test is evaluated once afterwards. Test accuracy is a reporting quantity, never a selection criterion. This is stated explicitly because a test-selected checkpoint on 56 images inflates results by several points.

The test set is small enough that this matters more than usual. 56 images means one image is **1.8 percentage points**. We therefore report 95% Wilson confidence intervals alongside point estimates, and we decline to declare a best architecture where those intervals overlap.

6. Results before hyperparameter tuning (baseline)

Executed on an NVIDIA Tesla T4 with mixed precision: 4 models $\times$ 60 epochs, batch 32, Adam $1e-4$, no weight decay, seed 42, best-validation checkpointing.

Model	Test accuracy	Macro F1	Macro spec.	Macro prec.	Best ep	95% CI (Wilson)
ResNet18	0.8036 (45/56)	0.8018	0.9608	0.8093	30	0.682–0.887
VGG16	0.7679 (43/56)	0.7667	0.9536	0.7725	4	0.642–0.859
ResNet34	0.7321 (41/56)	0.7303	0.9464	0.7314	2	0.604–0.830
D_ResNet (=ResNet50 approx.)	0.6964 (39/56)	0.6964	0.9393	0.7043	8	0.567–0.801

Table 6.1 — BASELINE results, before any hyperparameter tuning. Single seed (42), 60 epochs. This is the honest starting point; Table 6.2 shows how much of it survives repetition.

6.1 Paper versus our baseline

Model	Paper acc.	Paper prec.	Paper recall	Our acc.	Our macro F1	Gap
VGG16	0.83	0.89	0.83	0.7679	0.7667	-0.0621
ResNet18	0.78	0.87	0.78	0.8036	0.8018	+0.0236
ResNet34	0.83	0.89	0.83	0.7321	0.7303	-0.0979
D_ResNet (=ResNet50 approx.)	0.89	0.90	0.89	0.6964	0.6964	-0.1936

Table 6.2 — Paper-reported values beside our baseline. The comparison is NOT like-for-like; Section 10 explains why, and this table must not be read as a like-for-like benchmark.

We do not exceed the paper on accuracy for any model. Our ResNet18 (0.8036) and VGG16 (0.7679) fall short of the paper's 0.89 and 0.83 respectively. Section 10 sets out the reasons, the most important of which is that **our test set is strictly harder** because the paper's includes augmented images and ours does not.

6.2 Why we do not claim an architecture ranking

- **The intervals overlap.** Best (ResNet18, 0.682–0.887) and weakest (D_ResNet, 0.567–0.801) differ by 6 test images; the intervals overlap heavily.
- **The gap is at the noise floor.** $0.8036 - 0.6964 = 0.107$, or about 6 images out of 56. One image is 1.8 points.
- **No variance estimate existed yet.** These are single-seed numbers, so no error bar on the *ranking* can be computed. Section 8 supplies it, and the answer is that the ranking is not resolvable.

7. Hyperparameter tuning: design

7.1 What we tuned, and what we deliberately did not

The baseline is a **completely unregularised** model: weight decay 0.0, no label smoothing, and no explicit dropout. On a dataset where training accuracy reaches 1.000 by epoch 2, that is precisely the configuration in which regularisation should matter, so it is the most physically-motivated lever available rather than an arbitrary grid search.

Dropout was deliberately excluded as a tuning knob. torchvision's VGG16 already contains two Dropout (0.5) layers, while the ResNets contain none. Adding a dropout parameter would therefore *reduce* regularisation for VGG16 and *add* it for the ResNets, biasing any comparison between architectures. Weight decay and label smoothing apply identically to all four, so the cross-architecture comparison stays fair.

Configuration	Weight decay	Label smoothing	Role
A_baseline	0.0	0.0	Identical to Section 6; acts as the control
B_reg	1e-4	0.1	Candidate regularised configuration

Table 7.1 — The two configurations compared. Running the baseline configuration across three seeds is what supplies the error bar.

7.2 Trial matrix and budget

Dimension	Values	Count
Architectures	VGG16, ResNet18, ResNet34, D_ResNet	4
Configurations	A_baseline, B_reg	2
Seeds	42, 43, 44	3
Total trials	$4 \times 2 \times 3$	24
Epochs per trial	20	20
Total epochs trained	24×20	480
Measured GPU time	1.81 h total on one T4	—

Table 7.2 — Trial matrix. 20 epochs rather than 60 was justified by the epoch-2 saturation finding in Table 4.3.

7.3 The leakage guard: how the test set is protected

This is the design decision that determines whether the study means anything.

- **Within a trial:** the checkpoint is selected by highest validation accuracy. Test is not read during training.
- **After selection:** test is evaluated **once per trial** and reported.
- **Configuration choice:** made on **validation mean** across three seeds, and it stands on validation whatever test reports.

Evaluating test for all 24 trials is not leakage. Reporting 24 numbers is a statistic; *choosing the maximum of 24 test scores* would be leakage. The distinction is selection, not evaluation — and this report makes no test-based selection.

Resume capability. Each trial is appended to results/tuning/trials.csv immediately on completion, and finished trials are skipped on re-run. A reclaimed cloud runtime therefore costs at most one trial of work, which is exactly what happened once during this project.

8. Results after hyperparameter tuning

24 completed trials. Mean $\pm$ standard deviation across three seeds, plus the validation mean on which the configuration decision was actually based.

Model	A: val mean $\pm$ sd	A: test mean $\pm$ sd	B: val mean $\pm$ sd	B: test mean $\pm$ sd	Δ test
ResNet18	0.8788 $\pm$ 0.0278	0.7679 $\pm$ 0.0179	0.8727 $\pm$ 0.0182	0.8036 $\pm$ 0.0357	+0.0357
VGG16	0.8485 $\pm$ 0.0105	0.7143 $\pm$ 0.0179	0.8485 $\pm$ 0.0278	0.7321 $\pm$ 0.0179	+0.0179
ResNet34	0.8182 $\pm$ 0.0315	0.7321 $\pm$ 0.0472	0.8485 $\pm$ 0.0105	0.7619 $\pm$ 0.0273	+0.0298
D_ResNet (=ResNet50 approx.)	0.8364 $\pm$ 0.0182	0.7143 $\pm$ 0.0309	0.8424 $\pm$ 0.0105	0.7857 $\pm$ 0.0309	+0.0714

Table 8.1 — POST-TUNING results. Every model improved on test, but see Section 8.2: none of the gains exceeds its own seed spread.

8.1 Per-trial detail

Model	Config	seed 42	seed 43	seed 44	mean best ep
ResNet18	A_baseline	0.7500	0.7679	0.7857	7.0
ResNet18	B_reg	0.8393	0.7679	0.8036	10.0
VGG16	A_baseline	0.6964	0.7143	0.7321	7.0
VGG16	B_reg	0.7321	0.7143	0.7500	7.0
ResNet34	A_baseline	0.6786	0.7679	0.7500	11.0
ResNet34	B_reg	0.7857	0.7679	0.7321	7.0
D_ResNet (=ResNet50 approx.)	A_baseline	0.6964	0.7500	0.6964	9.0
D_ResNet (=ResNet50 approx.)	B_reg	0.8036	0.7500	0.8036	2.0

Table 8.2 — All 24 individual trial accuracies. Note how much a single seed moves a given architecture's score.

8.2 Why accuracy increased — and whether that increase is real

Accuracy did increase, on every architecture, in the same direction:

Configuration	Mean val (12 trials)	Mean test (12 trials)
A_baseline	0.8455	0.7321 $\pm$ 0.0349
B_reg	0.8530	0.7708 $\pm$ 0.0372
Δ	+0.0075	+0.0387

Table 8.3 — Configuration-level aggregates. Test rose roughly five times more than validation.

Three observations, taken together:

- **The direction is consistent.** All four architectures improved on test under B_reg. Random noise would not usually align across four independent models, so there is plausibly a small real effect.
- **Validation disagrees.** Validation moved +0.0075 while test moved +0.0387, a ratio of about five to one. If regularisation genuinely improved generalisation, validation should have moved comparably — it is computed from the same models. A large test-only gain points to seed luck.
- **The gain is inside the noise.** The configuration-level standard deviation is 0.0349 (A) and 0.0372 (B). The +0.0387 improvement is barely larger than a single configuration's own seed spread, and per-model each Δ sits inside its own seed spread.

Decisive test. A configuration counts as better only if its mean *validation* accuracy exceeds the baseline's mean by more than the baseline's own seed-to-seed standard deviation:

Model	A val mean	A sd	B val mean	Δ val	Verdict
D_ResNet	0.8364	0.0182	0.8424	+0.0061	inside spread — INCONCLUSIVE
ResNet18	0.8788	0.0278	0.8727	-0.0061	inside spread — INCONCLUSIVE

Model	A val mean	A sd	B val mean	Δ val	Verdict
ResNet34	0.8182	0.0315	0.8485	+0.0303	inside spread — INCONCLUSIVE
VGG16	0.8485	0.0105	0.8485	+0.0000	inside spread — INCONCLUSIVE

Table 8.4 — Selection metric is VALIDATION. The result: inconclusive for every architecture.

Conclusion: regularisation is *not* demonstrated to have helped. The improvement is consistent in sign but falls within seed noise, and the validation evidence that should corroborate it is nearly flat. We report this as a null result rather than claiming a tuning win, because the alternative would be reporting the largest of 24 test numbers as though it were a finding.

8.3 The finding that changed our reading of Section 6

The most valuable product of the tuning study is not the tuning result — it is the error bar on the baseline. Repeating the *identical* configuration across three seeds shows how much of Section 6 was luck:

Model	seed 42 (Section 6)	mean of 3 seeds	sd	movement (test images)
D_ResNet	0.6964	0.7143	0.0309	1.7
ResNet18	0.8036	0.7679	0.0179	1.0
ResNet34	0.7321	0.7321	0.0472	2.6
VGG16	0.7679	0.7143	0.0179	1.0

Table 8.5 — The seed-spread correction.

Both leading models in Section 6 sat above their own three-seed means. ResNet18's reported 0.8036 is the best of 0.7679 ± 0.0179 (three trials: 0.7500, 0.7679, 0.7857). VGG16's reported 0.7679 is well above its 0.7143 ± 0.0179 mean. The Section 6 ranking therefore looks modestly better than the underlying evidence supports — both models were reported at or near their luckiest draw.

Equally, ResNet34 moves by up to **2.6 test images** across seeds (sd 0.0472). Any architecture gap smaller than roughly 2–3 images was never resolvable from a single-seed run. This reaches the same conclusion as the overlapping Wilson intervals in Section 6, now backed by measured seed variability rather than by interval arithmetic alone.

8.4 Limitations of the tuning study

- **Aggregate accuracy only.** Notebook 04 recorded test accuracy per trial but no per-class reports, confusion matrices or checkpoints. Tuned models therefore cannot be broken down by class; only the baseline has that detail.
- **One split, three seeds.** These seeds measure *training* variability on a single fixed split. They do not measure *split* variability, which for 256 unique training images is likely the larger of the two.
- **20 epochs vs 60.** Comparable because best epochs were early, but the baseline column is not a re-trained 20-epoch control.
- **Two configurations, not a grid.** Deliberate: on 56 test images a wider search selects noise rather than signal.
- **Carried confound.** The background/class correlation from Section 2.3 applies unchanged to every tuned model.
- **No architecture ranking is claimed** from either the baseline or the tuning study.

9. Why our accuracy is lower than the paper's

Our best baseline (0.8036) is below the paper's D-ResNet (0.89). The gap is real, but attributing it to our models would be wrong. Six factors contribute, and they are listed in rough order of importance.

9.1 Our test set is strictly harder

This is the dominant methodological difference and it favours the paper's numbers. The paper augments before splitting, so its test set contains **rotated and flipped variants** of images that also appear in training. A model can recognise a test image because it has seen the same photograph in a different orientation. Our test set contains only clean, never-augmented, never-seen source images, so we measure genuine generalisation. **A meaningful part of the gap is the paper's measurement being more permissive, not our models being worse.**

9.2 The evaluation set is tiny

56 test images for the paper's single split (their count would be larger given augmentation). At this scale a single image shifts accuracy by 1.8 points, and the difference between 0.80 and 0.89 is about five images. Reported accuracies at this sample size carry wide confidence intervals, and ours do too (Table 6.1).

9.3 D_ResNet is an approximation, not a reproduction

As stated in Section 4.1, the paper's D-ResNet source and hyperparameters are unavailable, so our D_ResNet is torchvision ResNet50. Our D_ResNet scored 0.6964 against the paper's 0.89 — a gap of roughly 0.19, or about 11 test images. **We cannot attribute that gap to architecture quality, because we did not run their architecture.** The only paper-faithful comparisons available to us are VGG16, ResNet18 and ResNet34, and on those our gaps are 0.06, 0.02 and 0.10.

9.4 Hyperparameters were not available

The paper's optimiser, learning rate, batch size and regularisation settings are not published. We used Adam 1e-4, batch 32 and no weight decay as documented engineering choices (Table 4.2). A different learning rate or schedule can move accuracy by several points on a dataset this small, and we had no way to match settings we could not see.

9.5 Our split differs structurally

Our 256/55/56 source-level split produces a validation set of 55 images — enough to select a checkpoint, but too small to select hyperparameters reliably. The tuning study in Section 8 demonstrates exactly this: the selection metric moved 0.0075 while test moved 0.0387. On 55 validation images, selection is itself noisy.

9.6 A background confound inflates both sets equally — but inflates ours less

The background/class correlation (Cramer's $V \neq 0.442$) offers a shortcut to high accuracy. Both the paper and we benefit from it. It cannot explain a *gap*, but it does mean both figures are upper bounds, and it means our absolute numbers should not be read as mineral-identification accuracy in the field.

9.7 What we can and cannot conclude

Statement	Supported?
Our best baseline accuracy is 0.8036 (ResNet18, seed 42, 60 epochs)	Yes — measured, with a Wilson CI
Regularisation improved our models beyond seed noise	No — inside seed spread, validation flat
Our architectures are worse than the paper's	Not established — different, harder test protocol
We exactly reproduced D-ResNet	No — approximation; explicitly disclaimed
The paper's results are wrong	Not established — different protocol, no re-analysis possible
The architecture ranking is resolvable from this dataset	No — intervals overlap and seed spread is 2–3 images

Table 9.1 — Defensible claims. The safest summary of this project is that it reproduces the experimental setting and quantifies the noise floor, not that it outperforms or falsifies the paper.

Honest summary. Our gap to the paper is largely explained by a stricter, leakage-free evaluation protocol, an unavailable architecture, unpublished hyperparameters, and a 56-image test set. A meaningful residual gap probably remains, but isolating it would require the paper's code and settings. We would not claim our models are simply worse, and we would not claim to have matched the paper.

10. Anticipated examiner questions

Prepared answers to the questions this presentation is most likely to draw.

Q. Why is your accuracy lower than the paper's?

Principally because our test set is harder. The paper augments before splitting, so rotated and flipped copies of training photographs appear in their test set; ours contains only clean, never-augmented images that appear in no split. Additionally, our D_ResNet is a ResNet50 approximation, since the original implementation is unavailable; our hyperparameter search differs from theirs as their settings are unpublished; and at 56 test images a few images move accuracy 1.8 points each. Section 9 addresses each factor separately.

Q. Why did you change the split order from the paper?

Augmenting before splitting leaks. Rotated and flipped copies of one photograph land in train and test, so a model is scored on images it has already seen, and the result measures memorisation rather than generalisation. We split at the source-image level first, then augmented the training split only, and we automated a leakage check that re-verifies the disjointness at training time. This is a deliberate, documented deviation and it makes our numbers conservative.

Q. Your D_ResNet is not the paper's D_ResNet. Why claim it?

We do not claim it. It is explicitly labelled an approximation throughout, and Section 4.1 disclaims exact reproduction. We used it to test whether a deeper bottleneck residual network helps at this scale, which it did not in our hands. Every table marks it as ResNet50.

Q. How many epochs did you train, and why?

The baseline ran the full 60 epochs per model as the paper specifies, 240 epochs across four models, in 56 minutes on one T4. Training accuracy reached 1.000 by epoch 2 and three of four models peaked in validation by epoch 30. Because the useful training finished early, the tuning trials used 20 epochs — 480 epochs across 24 trials — which was defensible and cut compute roughly threefold.

Q. What are your parameter counts?

VGG16 134.29 M, ResNet18 11.18 M, ResNet34 21.29 M, and ResNet50 23.52 M. All verified at runtime by the model-construction step in notebook 02. VGG16 is by far the largest and also the slowest, at 25.8 seconds per epoch against 7.1 for ResNet18.

Q. Your tuning did not clearly help. Was it worth running?

Yes, though not for the reason expected. The regularisation gain of +0.0387 sits inside the seed spread of 0.0349, and validation moved only +0.0075, so we report it as not demonstrated. What the study bought was an error bar on the baseline: seed spread of 0.018 to 0.047 revealed that the single-seed ranking in the baseline was never resolvable, and that two of our reported figures sat above their own three-seed means.

Q. Why only 56 test images? Why not a larger split?

The dataset contains 367 images total, so a 15% test split is 56 images. That is a property of the dataset, not our choice. It is the main reason we report confidence intervals and decline to rank architectures: one image is worth 1.8 percentage points and the entire accuracy range across models is about six images.

Q. Which model is the best?

We do not claim one. ResNet18 has the highest point estimate at 0.8036, but its 95% Wilson interval of 0.682 to 0.887 overlaps every other model's, and the seed study shows the ranking moves by two to three images between seeds. Declining to name a winner is the statistically defensible position on this data.

Q. How do you know you did not overfit the test set?

Checkpoints were selected by validation accuracy, never test. In the tuning study test was read once per trial after validation-based checkpoint selection, and the configuration decision was made on validation mean and reported as such. We never selected the maximum of the 24 test scores; that would have been leakage.

Q. What is the largest limitation of this work?

Background composition correlates with class at Cramer's V of roughly 0.442, so a CNN can learn background colour instead of grain morphology. Every accuracy figure here is an upper bound on genuine mineral identification. This carried from a separate audit and was not re-measured inside this project, so we report it as a documented risk.

Q. Why did you use augmentation only on train?

Because augmenting validation and test would make model selection and final evaluation meaningless: the same photograph would appear in multiple splits in different orientations. Our validation and test sets contain only clean source images.

Q. Can you demonstrate the results are reproducible?

Yes. Notebook 01 writes SHA-256 fingerprints of every manifest plus full library versions. All five notebooks are committed in executed form with outputs intact, so the code carries its own evidence of having run. Checkpoints are stored on cloud storage and deliberately never committed.

Q. What would you do next?

First, re-measure the background confound and report accuracy on a background-normalised or background-masked subset, which is the only way to know whether the models learned morphology. Second, increase the number of seeds to roughly ten and use repeated stratified group splits to separate training variance from split variance. Third, acquire an independent labelled test set of beach-sand samples, since 56 images is the binding constraint on every conclusion above.

Q. Why is your dataset total 367 when the paper says 357?

The paper's text says 357 but its own six per-class counts sum to 367, and several labels appear misassigned relative to those counts. We report the on-disk truth and never force a total to match the published figure.

11. Conclusion and future work

This project reproduced the experimental setting of a published BSM classification study across four architectures and delivered two defensible results.

- **A baseline of 0.8036 test accuracy (ResNet18, 60 epochs, seed 42)**, with the caveat that this figure sits at the top of its own seed distribution and the mean across three seeds is 0.7679 ± 0.0179 .
- **A measured noise floor.** Seed-to-seed standard deviation of 0.018 to 0.047 on a 56-image test set, which establishes that the architecture ranking in this dataset is not resolvable. That is a negative result, and a useful one: it replaces an unsupported ranking with a quantified uncertainty.
- **A null tuning result, stated as such.** Regularisation improved every model in the same direction but by less than the seed spread, and validation barely moved. Reported honestly rather than presented as a win.

The overall lesson is that with 367 images and 56 test images, measurement precision is the binding constraint, not model capacity. The most valuable thing this project produced is not an accuracy figure but an honest account of how much of that figure can be trusted — including the parts that turned out not to be.

11.1 Future work, in priority order

- **Quantify the background shortcut.** Re-measure accuracy on background-normalised or background-masked images. Until this is done, no absolute number in this report should be interpreted as mineral-identification performance.
- **Separate training variance from split variance.** Use repeated stratified group splits across at least ten seeds, rather than three seeds on one split.

- **Acquire an independent test set.** A few hundred genuinely unseen beach-sand samples would narrow every confidence interval in this report.
- **Address near-duplicates.** Prior analysis of this dataset found 27 near-duplicate pairs in 23 clusters, four of which span classes, and the current split guarantees exact-source grouping but not near-duplicate grouping.
- **Recover per-class tuned metrics.** Re-run a small tuning sweep that saves classification reports and checkpoints, closing the aggregate-only limitation in Section 8.4.

Report generated from committed artefacts: results/metrics/final_results.csv, results/metrics/paper_reference_values.csv, results/tuning/trials.csv, results/tuning/per_config_summary.csv, results/tuning_vs_baseline/baseline_vs_tuned.csv, results/dataset_summary/dataset_summary.csv, results/preprocessing/dataset_version_summary.csv. All figures in this document are read from those files at generation time rather than transcribed by hand.